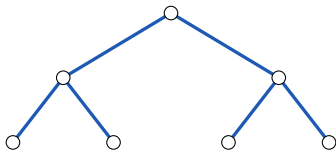


Trees

Section 1.3

Harris–Hirst–Mossinghoff, *Combinatorics and Graph Theory*



Jeremy Quastel

Today's lecture

1. Definitions and Examples

Section 1.3.1.

2. Properties of Trees

Section 1.3.2.

3. Spanning Trees

Section 1.3.3.

4. Counting Trees

Section 1.3.4.

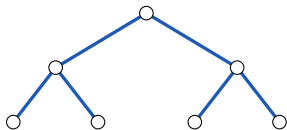
Definition: Trees and Forests

Tree

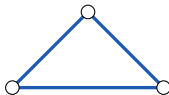
A **tree** is a connected graph with no cycles.

Forest

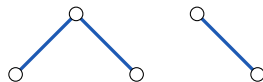
A **forest** is a graph whose connected components are trees. Equivalently, it is an acyclic graph.



A: A tree



B: Contains a cycle

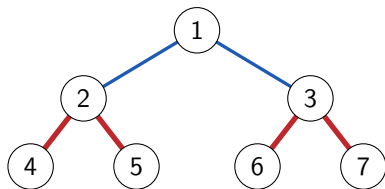


C: A forest with two components

Definition: Leaves

Leaf

A vertex of degree 1 in a tree is called a **leaf**.



The leaves are 4, 5, 6, 7.

K_1 is a tree, but its only vertex has degree 0 and is not a leaf.

Types of Trees of Order 6 or less

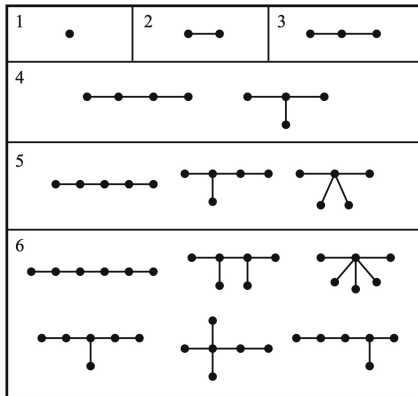


FIGURE 1.31. Trees of order 6 or less.

Source: Harris, Hirst, and Mossinghoff, *Combinatorics and Graph Theory*, 2nd ed., Springer, 2008, p. 32, Figure 1.31.

Application 1: Possible Outcomes

A branching diagram records the possible results of successive steps in an experiment, such as tossing a coin and rolling a die.

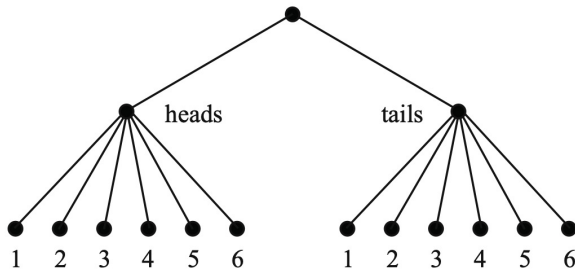


FIGURE 1.32. Outcomes of a coin/die experiment.

Source: Harris, Hirst, and Mossinghoff, *Combinatorics and Graph Theory*, 2nd ed., Springer, 2008, p. 32, Figure 1.32.

Application 2: Computer Algorithms

Trees organize searches and sorting procedures, and represent the sequence of decisions made by an algorithm.

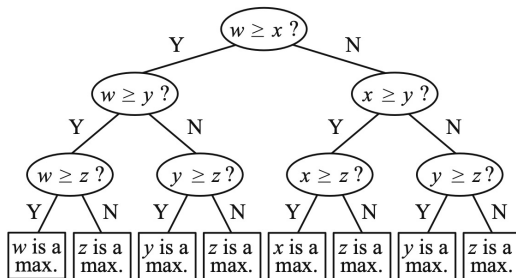


FIGURE 1.33. Logic of a program.

Source: Harris, Hirst, and Mossinghoff, *Combinatorics and Graph Theory*, 2nd ed., Springer, 2008, p. 33, Figure 1.33.

Application 3: Chemical Compounds

Trees can model the atoms and bonds in acyclic saturated hydrocarbons. Carbon atoms have degree 4, and hydrogen atoms are leaves.

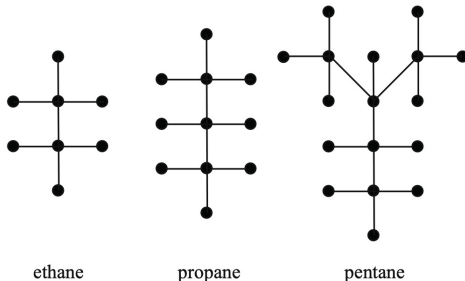


FIGURE 1.34. A few saturated hydrocarbons.

Source: Harris, Hirst, and Mossinghoff, *Combinatorics and Graph Theory*, 2nd ed., Springer, 2008, p. 33, Figure 1.34.

Application 4: Tournament Brackets

A tree records how winners advance through successive rounds of a single-elimination tournament.

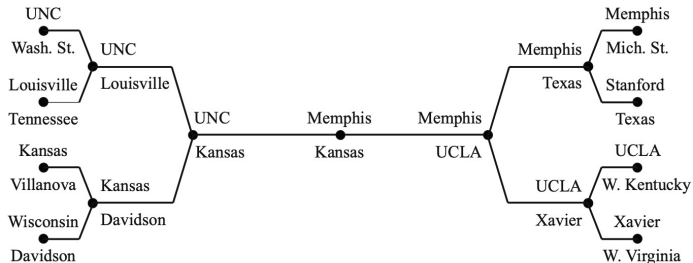


FIGURE 1.35. The 2008 Men's Sweet 16.

Source: Harris, Hirst, and Mossinghoff, *Combinatorics and Graph Theory*, 2nd ed., Springer, 2008, p. 33, Figure 1.35.

Question

Draw a tree with 10 vertices. How many edges does it have?

Does the answer depend on the shape of your tree?

A Useful Observation: Unique Paths

Unique paths

In a tree, each pair of distinct vertices is joined by exactly one path.

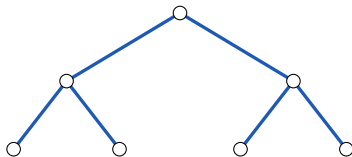
- 1 Connectedness gives at least one path.
- 2 If there were two different paths, take a point where they separate and their first subsequent meeting. The two segments form a cycle.

Consequence. Every edge of a tree is a bridge. Deleting it leaves exactly two components.

Theorem: Counting Edges

Theorem 1.10

A tree with n vertices has exactly $n - 1$ edges.



7 vertices, 6 edges.

Proof of Theorem 1.10

Theorem 1.10

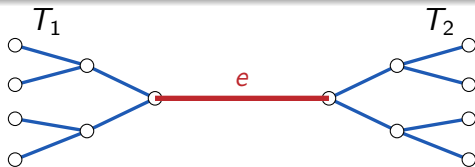
A tree with n vertices has exactly $n - 1$ edges.

Proof

Induct on n . For $n = 1$, the tree is K_1 and has 0 edges. Assume the result for all smaller trees, and let T have $n \geq 2$ vertices.

Delete any edge e . Since T has no cycles, $T - e$ consists of two trees T_1, T_2 , with orders $n_1, n_2 < n$ and $n_1 + n_2 = n$. By induction, they have $n_1 - 1$ and $n_2 - 1$ edges. Restoring e gives

$$|E(T)| = (n_1 - 1) + (n_2 - 1) + 1 = n - 1. \quad \square$$



Theorem: Counting Edges in a Forest

Theorem 1.11

A forest with n vertices and k components has $n - k$ edges.

Proof. If the components have orders $n_1, \dots, n_k$, then

$$|E(F)| = \sum_{i=1}^k (n_i - 1) = n - k. \quad \square$$

Example

A forest with 12 vertices and 9 edges has 3 components.

Theorem: Connectedness and Edge Count

Theorem 1.12

A graph on n vertices is a tree if and only if it is connected and has $n - 1$ edges.

Theorem: Connectedness and Edge Count

Theorem 1.12

A graph on n vertices is a tree if and only if it is connected and has $n - 1$ edges.

Proof

Forward direction. A tree is connected by definition and has $n - 1$ edges by Theorem 1.10.

Reverse direction. Suppose G is connected and has $n - 1$ edges. If G contains a cycle, delete one edge from that cycle. This preserves connectedness.

Continue deleting cycle edges until no cycle remains. The resulting graph is a tree on the same n vertices. If any edge was deleted, this tree has fewer than $n - 1$ edges, contradicting Theorem 1.10.

Thus G is acyclic and connected, so it is a tree. □

Theorem: Acyclicity and Edge Count

Theorem 1.13

A graph on n vertices is a tree if and only if it is acyclic and has $n - 1$ edges.

Theorem: Acyclicity and Edge Count

Theorem 1.13

A graph on n vertices is a tree if and only if it is acyclic and has $n - 1$ edges.

Proof

Forward direction. A tree is acyclic by definition and has $n - 1$ edges by Theorem 1.10.

Reverse direction. Suppose G is acyclic and has $n - 1$ edges. Since G is acyclic, it is a forest. Let k be its number of components.

By Theorem 1.11, a forest with n vertices and k components has $n - k$ edges. Hence

$$n - 1 = |E(G)| = n - k,$$

so $k = 1$. Thus G is connected and acyclic, and therefore is a tree. □

Theorem: Trees Have Leaves

Theorem 1.14

Every tree with at least two vertices has at least two leaves.

Theorem: Trees Have Leaves

Theorem 1.14

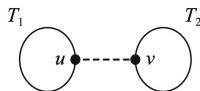
Every tree with at least two vertices has at least two leaves.

Proof

Induct on the number n of vertices. For $n = 2$, both vertices are leaves. Suppose $n \geq 3$ and the result holds for smaller trees.

Case 1: Every edge meets a leaf. Choose two distinct edges. Their leaf endpoints are distinct, giving two leaves.

Case 2: Some edge uv has no leaf endpoint. Delete uv . The resulting trees T_1, T_2 each have at least two vertices and fewer than n . By induction, each has at least two leaves. Choose a leaf of T_1 other than u and a leaf of T_2 other than v . Both remain leaves when uv is restored. $\square$



Another Proof: A Longest Path

Choose a longest path $v_0, v_1, \dots, v_r$ in the tree.



- 1 If v_0 had a neighbor outside the path, we could extend it.
- 2 If v_0 had a neighbor on the path other than v_1 , there would be a cycle.

So v_0 is a leaf. The same argument applies to v_r .



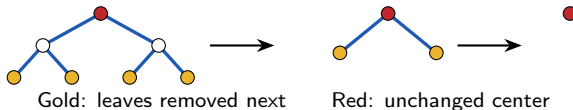
Theorem: The Center of a Tree

Theorem 1.15

The center of a tree consists of one vertex or two adjacent vertices.

Proof idea

Removing all leaves of a tree with at least three vertices does not change its center. Repeat until only one vertex or two adjacent vertices remain.



Theorem: The Center of a Tree

Theorem 1.15

The center of a tree consists of one vertex or two adjacent vertices.

Proof

The result is immediate for K_1 and K_2 . Let T have at least three vertices, and remove all its leaves to obtain a nonempty tree T' .

No leaf is central: its neighbor has smaller eccentricity. For each remaining vertex v , every longest path starting at v ends at a leaf; pruning shortens its maximum length by exactly one. Hence $\text{ecc}_{T'}(v) = \text{ecc}_T(v) - 1$. All remaining eccentricities decrease equally, so T and T' have the same center. Repeat while there are at least three vertices. The process ends at K_1 or K_2 , whose one or two adjacent vertices form the original center. $\square$



Theorem: Finding Trees Inside Other Graphs

Theorem 1.16

Let T be a tree with k edges. Every nonempty graph G with $\delta(G) \geq k$ contains a subgraph isomorphic to T .

Equivalently, G contains every tree on at most $\delta(G) + 1$ vertices.

Remark

The copy need not be an induced subgraph. We may leave out extra edges of G .

Theorem: Finding Trees Inside Other Graphs

Theorem 1.16

Let T be a tree with k edges. Every nonempty graph G with $\delta(G) \geq k$ contains a subgraph isomorphic to T .

Theorem: Finding Trees Inside Other Graphs

Theorem 1.16

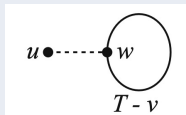
Let T be a tree with k edges. Every nonempty graph G with $\delta(G) \geq k$ contains a subgraph isomorphic to T .

Proof

Induct on k . For $k = 0$, any vertex of G gives a copy of T . Suppose $k \geq 1$ and the result holds for trees with $k - 1$ edges.

Choose a leaf v of T with neighbor w . By induction, G contains a copy of $T - v$, which has k vertices. Identify w with its image in this copy.

Since $\deg_G(w) \geq k$ and only $k - 1$ other vertices lie in the copy, w has a neighbor u outside it. Adding u and the edge uw gives a subgraph isomorphic to T , with u representing v . $\square$



Definition: Spanning Tree

Spanning tree

A **spanning tree** of G is a subgraph T that is a tree and satisfies $V(T) = V(G)$.

Every connected graph has a spanning tree: delete cycle edges until no cycle remains.

A spanning tree on n vertices always has $n - 1$ edges.

A disconnected graph has no spanning tree.

Example: Planning Railroad Track

A railroad network must connect all eight cities shown. The edges represent possible track links, with a weight assigned to each link.

We want every city to be reachable from every other city while minimizing the total weight of the selected links.

A cycle would be unnecessary: removing one of its edges preserves connectivity and reduces the total weight.

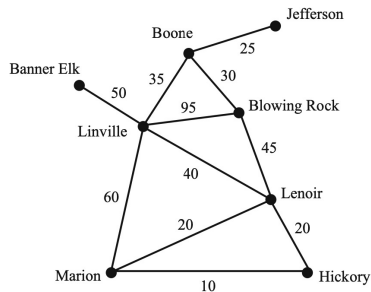


FIGURE 1.41. The city graph.

Goal

Find a **minimum weight spanning tree** of the city graph.

Definition: Minimum Weight Spanning Tree

Weighted graph

A weight function assigns a number $w(e) \geq 0$ to each edge e .

Minimum weight spanning tree

A spanning tree T is a **minimum weight spanning tree** if it minimizes

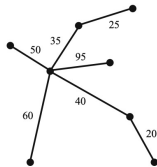
$$w(T) = \sum_{e \in E(T)} w(e)$$

among all spanning trees of G .

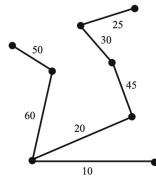
Different spanning trees can achieve the same minimum.

Examples: Spanning Trees of the City Graph

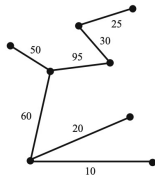
Question. Is one of these a minimum weight spanning tree?



Total Weight = 325



Total Weight = 240



Total Weight = 290

Figure source: Harris, Hirst, and Mossinghoff, *Combinatorics and Graph Theory*, 2nd ed., Springer, 2008, §1.3.3, spanning trees of the city graph.

Kruskal's Algorithm

There are a number of fairly simple algorithms for finding minimum weight spanning trees. Perhaps the best known is Kruskal's algorithm.

Algorithm

- 1 Start with all vertices and no chosen edges.
- 2 Consider the edges in order of increasing weight.
- 3 Add an edge if it creates no cycle with the chosen edges. Otherwise skip it.
- 4 Stop after choosing $n - 1$ edges.

Ties may be resolved in any order. The chosen edges form a forest at every step.

Kruskal's Algorithm for the Railroad Problem

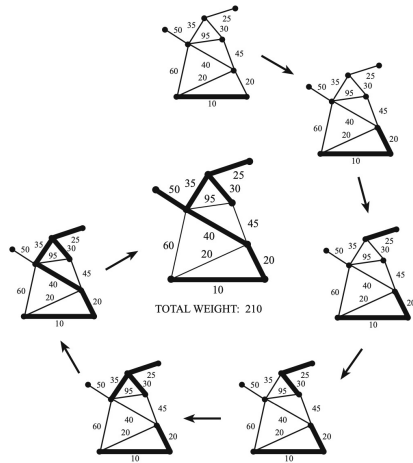
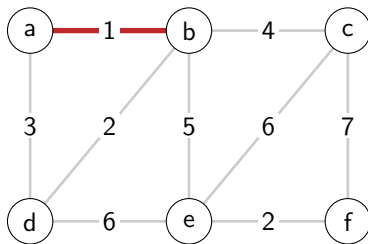


Figure source: Harris, Hirst, and Mossinghoff, *Combinatorics and Graph Theory*, 2nd ed., Springer, 2008, §1.3.3, Kruskal's algorithm for the city graph.

Kruskal's Algorithm: Step 1

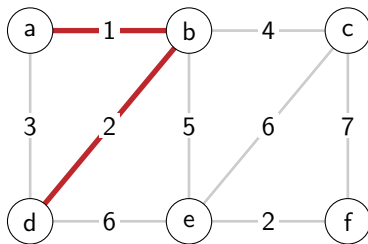
Choose ab of weight 1.



Chosen edges are red. Total weight: 1.

Kruskal's Algorithm: Step 2

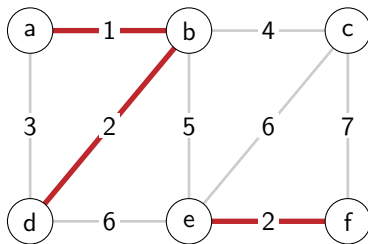
Choose *bd* of weight 2.



Chosen edges are red. Total weight: 3.

Kruskal's Algorithm: Step 3

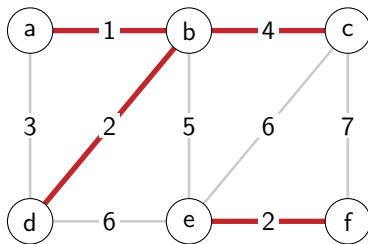
Choose ef of weight 2.



Chosen edges are red. Total weight: 5.

Kruskal's Algorithm: Step 4

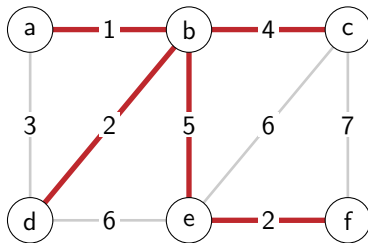
Skip ad of weight 3: it closes a cycle. Choose bc of weight 4.



Chosen edges are red. Total weight: 9.

Kruskal's Algorithm: Step 5

Choose *be* of weight 5. All six vertices are now connected.



Chosen edges are red. Total weight: 14.

Why Does Kruskal's Algorithm Produce a Tree?

- ① An edge is added only when it joins two different components, so no cycle is ever formed.
- ② If the chosen forest were still disconnected after all edges had been considered, connectedness of G would provide an edge between two components. That edge could not have been skipped.
- ③ Thus the algorithm reaches a connected forest: a spanning tree.

But why must its total weight be minimum?

Theorem: Kruskal's Algorithm Is Optimal

Theorem 1.17

Kruskal's algorithm produces a minimum weight spanning tree.

Theorem: Kruskal's Algorithm Is Optimal

Theorem 1.17

Kruskal's algorithm produces a minimum weight spanning tree.

Proof

Assume G is connected. Let T be the spanning tree produced by Kruskal's algorithm, with edges $e_1, \dots, e_{n-1}$ listed in the order chosen.

Among all minimum weight spanning trees of G , choose T' that agrees with T for as long as possible: the initial sequence $e_1, \dots, e_k$ contained in T' is as long as possible.

If $T' = T$, we are done. Otherwise, $0 \leq k < n - 1$ and

$$e_1, \dots, e_k \in E(T'), \quad e_{k+1} \notin E(T').$$

Proof continued

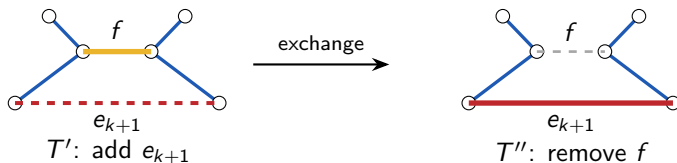
Proof of Theorem 1.17 (continued)

Adding e_{k+1} to T' creates a unique cycle C . Since $e_1, \dots, e_k, e_{k+1}$ form a forest, C contains another edge $f \in E(T')$ joining different components of the forest $F = \{e_1, \dots, e_k\}$. In particular, $f \notin F$.

The edge f was eligible when Kruskal chose e_{k+1} , so $w(e_{k+1}) \leq w(f)$. Replacing f by e_{k+1} gives a spanning tree

$$T'' = T' + e_{k+1} - f, \quad w(T'') = w(T') + w(e_{k+1}) - w(f) \leq w(T').$$

Thus T'' is also minimum, but contains $e_1, \dots, e_k, e_{k+1}$, contradicting the choice of T' . Hence T is a minimum weight spanning tree. $\square$



Another Algorithm: Prim's Algorithm

Algorithm

- ① Start at any vertex.
- ② Choose a cheapest edge joining a reached vertex to an unreached vertex.
- ③ Add the edge and the new vertex. Repeat until every vertex is reached.

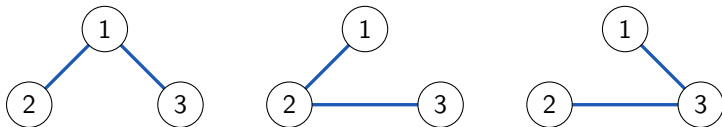
Kruskal grows a forest. Prim grows one connected tree.

Exercise. Apply Prim starting at a in our six-town example.

Counting Trees: Labels Matter

Labeled trees

The vertex labels are fixed. Changing which labeled vertices are joined can give a different tree, even if the two trees are isomorphic.



Three labeled trees on $\{1, 2, 3\}$, but only one shape.

Theorem: Cayley's Tree Formula

How many different trees can we form on n vertices with fixed labels? We count trees as different when their edges join different pairs of labels, even if their shapes are the same. Cayley's formula gives this number directly.

Theorem 1.18 (Cayley's Tree Formula)

There are n^{n-2} distinct labeled trees of order n .

n	2	3	4	5	6
Trees	1	3	16	125	1296

The case $n = 1$ has one tree, K_1 .

Proof idea. We prove the formula by establishing a one-to-one correspondence between labeled trees on $\{1, \dots, n\}$ and sequences of length $n - 2$ with entries in $\{1, \dots, n\}$. There are n^{n-2} such sequences, so there are n^{n-2} labeled trees.

The following slides present the two algorithms for going back and forth: from a tree to its sequence, and from a sequence to its unique tree.

Prüfer's Method for Assigning a Sequence to a Labeled Tree

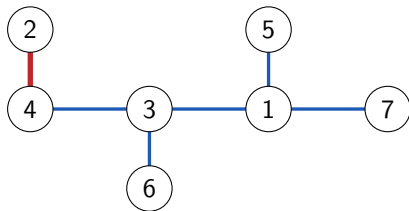
This algorithm assigns each labeled tree a sequence of length $n - 2$: each step records one label and removes one vertex, stopping when two vertices remain.

Algorithm

Given: A tree T , with vertices labeled $1, \dots, n$.

- 1 Let $i = 0$, and let $T_0 = T$.
- 2 Find the leaf on T_i with the smallest label and call it v .
- 3 Record in the sequence the label of v 's neighbor.
- 4 Remove v from T_i to create a new tree T_{i+1} .
- 5 If $T_{i+1} = K_2$, then stop. Otherwise, increment i by 1 and go back to step 2.

Prüfer Example: Step 1

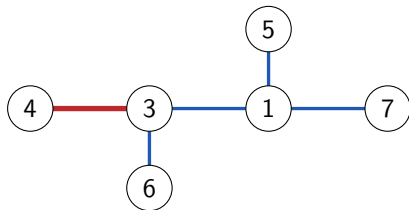


Smallest leaf: 2. Record its neighbor: 4.

Sequence so far

4

Prüfer Example: Step 2

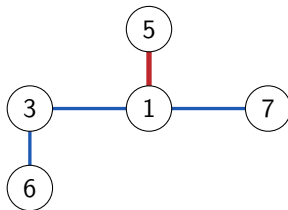


Smallest leaf: 4. Record its neighbor: 3.

Sequence so far

4, 3

Prüfer Example: Step 3

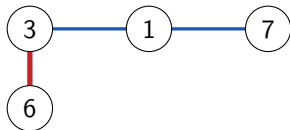


Smallest leaf: 5. Record its neighbor: 1.

Sequence so far

4, 3, 1

Prüfer Example: Step 4

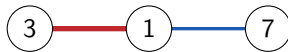


Smallest leaf: 6. Record its neighbor: 3.

Sequence so far

4, 3, 1, 3

Prüfer Example: Step 5



Smallest leaf: 3. Record its neighbor: 1.

Sequence so far

4, 3, 1, 3, 1

What Does the Prüfer Sequence Remember?

Degree formula

For every vertex v ,

$$\deg_T(v) = 1 + \#\{\text{occurrences of } v \text{ in the sequence}\}.$$

Each recorded occurrence removes one edge incident to v . Exactly one incident edge remains when v is deleted or reaches the final pair.

Consequently: the leaves are exactly the labels absent from the sequence.

Prüfer's Method for Constructing a Tree from a Sequence

We can reverse the procedure: every sequence of length $n - 2$ with entries in $\{1, \dots, n\}$ determines a unique labeled tree. The algorithm below reconstructs that tree, undoing the leaf-removal process.

Algorithm

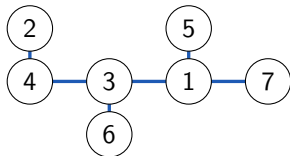
Given: A sequence $(a_1, \dots, a_{n-2})$, with each $a_i \in \{1, \dots, n\}$.

- 1 Start with vertices labeled $1, \dots, n$, no edges, and $S = \{1, \dots, n\}$.
- 2 Find the smallest label $b \in S$ that does not occur in the remaining sequence.
- 3 Join b to the vertex whose label is the first entry of the sequence.
- 4 Remove b from S and delete the first entry. If entries remain, return to step 2.
- 5 Join the two remaining vertices in S .

For $n = 2$, the sequence is empty: simply join vertices 1 and 2.

Decoding (4, 3, 1, 3, 1)

Remaining sequence	Smallest absent label	Add edge	Remove label
(4, 3, 1, 3, 1)	2	24	2
(3, 1, 3, 1)	4	43	4
(1, 3, 1)	5	51	5
(3, 1)	6	63	6
(1)	3	31	3
empty	1, 7 remain	17	—



Proof of Cayley's Formula

- 1 Decoding any sequence produces a tree: read the construction backward, starting with the final edge and attaching one new leaf at each step.
- 2 In each remaining tree, its leaves are precisely the remaining labels absent from the remaining sequence. Thus encoding and decoding select the same smallest leaf and the same neighbor.
- 3 The procedures are inverse, so trees correspond bijectively to sequences in $\{1, \dots, n\}^{n-2}$.

$$\# \text{trees} = \underbrace{n \cdot n \cdots n}_{n-2 \text{ entries}} = n^{n-2}. \quad \square$$

Questions About Prüfer Sequences

- ① Which tree corresponds to the constant sequence $(a, a, \dots, a)$?
- ② What can you say about a tree whose sequence entries are all distinct?
- ③ How many labeled trees on $n \geq 3$ vertices have vertex 1 as a leaf?

Questions About Prüfer Sequences: Answers

- ① A star with center a . Its center has degree $n - 1$, and all other degrees are 1.
- ② Every degree is at most 2, so the tree is a path.
- ③ The sequence must avoid 1. Each of its $n - 2$ entries has $n - 1$ choices, giving $(n - 1)^{n-2}$ trees.

Counting Spanning Trees: Toward the Matrix Tree Theorem

Cayley's formula counts labeled trees on n vertices. Equivalently, it counts the spanning trees of the complete graph K_n .

What if the graph is not complete? We would like a way to count spanning trees when only certain pairs of vertices are joined by edges.

The **Matrix Tree Theorem** answers this more general question. Proved by Kirchhoff in 1847, it connects the number of spanning trees of a graph to a determinant formed from its matrices.

Definitions: Degree Matrix and Cofactor

To state the Matrix Tree Theorem, we need two definitions. Recall that A denotes the adjacency matrix.

Degree matrix

For vertices $v_1, \dots, v_n$, define the $n \times n$ matrix D by

$$[D]_{i,j} = \begin{cases} \deg(v_i), & i = j, \\ 0, & i \neq j. \end{cases}$$

Thus D lists the vertex degrees on its diagonal and is zero elsewhere.

Cofactor

For an $n \times n$ matrix B , its (i, j) cofactor is $(-1)^{i+j} \det B(i | j)$, where $B(i | j)$ is obtained by deleting row i and column j .

Theorem: The Matrix Tree Theorem

Theorem 1.19 (Matrix Tree Theorem)

For a connected labeled graph G , let A be its adjacency matrix and D its degree matrix. Every cofactor of $D - A$ equals the number of spanning trees of G .

Matrix Tree Theorem: The Proof in Three Steps

Fix a vertex v_r , called the **root**. We first count trees using the (r, r) cofactor.

The plan

- 1 Write the matrix with row and column r deleted as BB^T , using a signed incidence matrix.
- 2 Expand $\det(BB^T)$ as a sum over sets of $n - 1$ edges.
- 3 Show that each spanning tree contributes 1, and every other edge set contributes 0.

Finally, we explain why all cofactors have the same value.

Step 1: Give Each Edge a Direction

Let G have n vertices and m edges. Choose a direction for each edge; these directions are only a bookkeeping device.

Signed incidence matrix M

The column for an edge $u \rightarrow v$ has -1 in row u , $+1$ in row v , and zeros elsewhere.



Thus M is $n \times m$. Reversing an edge multiplies its column by -1 .

Step 1: Recovering the Degree and Adjacency Matrices

Claim A

$$MM^T = D - A.$$

Proof: take dot products of rows

Diagonal entry: row i has one entry ± 1 for each edge at v_i . Its squared length is $\deg(v_i)$.

Off-diagonal entry: rows i and j overlap only in a column for an edge joining v_i and v_j . That column contributes $(-1)(+1) = -1$.

Hence the diagonal entries are those of D , and the off-diagonal entries are those of $-A$.

Step 1: Delete the Root Row

Delete row r of M to obtain the $(n - 1) \times m$ matrix B .

The cofactor we want

Deleting row and column r of MM^T leaves the dot products of the remaining rows. Therefore

$$(D - A)(r \mid r) = BB^T.$$

The (r, r) cofactor is consequently $\det(BB^T)$.

For $S \subseteq E(G)$ with $|S| = n - 1$, write B_S for the square matrix obtained by keeping the columns indexed by S .

Step 2: Expand the Determinant by Edge Sets

Cauchy–Binet formula

For a $q \times m$ matrix X and an $m \times q$ matrix Y ,

$$\det(XY) = \sum_{|S|=q} \det(X_{[:,S]}) \det(Y_{[S,:]}).$$

The sum runs over sets of q column indices, in increasing order.

Apply this with $X = B$, $Y = B^T$ and $q = n - 1$:

$$\det(BB^T) = \sum_{\substack{S \subseteq E(G) \\ |S|=n-1}} (\det B_S)^2.$$

Each term now corresponds to a spanning subgraph with $n - 1$ edges.

Step 3: Which Edge Sets Contribute?

Claim B

For $H = (V(G), S)$ with $|S| = n - 1$,

$$(\det B_S)^2 = \begin{cases} 1, & H \text{ is a spanning tree,} \\ 0, & H \text{ is not a spanning tree.} \end{cases}$$

Tree: remove a leaf other than the root and expand the determinant along its row.

Not a tree: find a component not containing the root; its rows sum to zero.

Claim B: A Tree Contributes One

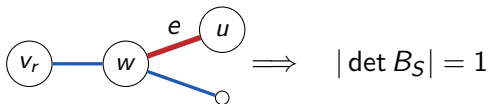
Proof by removing leaves

If H is a tree with at least two vertices, choose a leaf $u \neq v_r$ and let e be its only edge.

The row of B_S for u has just one nonzero entry, equal to ± 1 , in column e . Expanding along this row gives

$$|\det B_S| = |\det B_{\text{smaller tree}}|.$$

The smaller matrix is the signed incidence matrix of $H - u$, with the same root row deleted. Repeat until only the root remains; the empty determinant is 1.



Claim B: Every Other Edge Set Contributes Zero

Proof by finding dependent rows

An n -vertex graph with $n - 1$ edges is a tree if it is connected. Thus, if H is not a tree, it is disconnected.

Choose a component C not containing v_r . All of its vertex rows remain in B_S .

Sum these rows. An edge in C contributes $-1 + 1 = 0$; an edge outside C contributes 0. No edge of H leaves C .

The sum is the zero row, so the rows are dependent and $\det B_S = 0$.

This also covers an isolated vertex: its row is already zero.

Putting the Pieces Together

The diagonal cofactor counts spanning trees

$$\begin{aligned}\det((D - A)(r \mid r)) &= \det(BB^T) \\ &= \sum_{\substack{S \subseteq E(G) \\ |S|=n-1}} (\det B_S)^2 \\ &= \sum_{\substack{S \subseteq E(G) \\ (V(G), S) \text{ is a tree}}} 1.\end{aligned}$$

Every spanning tree has exactly $n - 1$ edges and appears once.

The root v_r was arbitrary, so this proves the result for every diagonal cofactor.

Why Every Cofactor Has the Same Value

Set $L = D - A$ and $\mathbf{1} = (1, \dots, 1)^T$. Since $L\mathbf{1} = 0$, we have $\det L = 0$.

The only vectors in the kernel are constant

If $Lx = 0$, then

$$0 = x^T Lx = \|M^T x\|^2 = \sum_{uv \in E(G)} (x_u - x_v)^2.$$

Thus $x_u = x_v$ along every edge. Connectedness implies x is constant.

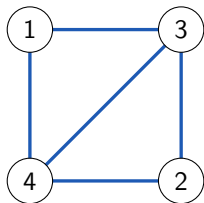
Apply the cofactor identity

Let C_{ij} be the (i, j) cofactor and let $Q_{ij} = C_{ji}$, the adjugate. Cofactor expansion gives

$$LQ = (\det L)I = 0.$$

Each column of Q is therefore constant. Its j th entry is $Q_{jj} = C_{jj}$, which we proved equals the number of spanning trees. Hence every entry of Q , and every cofactor, equals this number. $\square$

Matrix Tree Theorem: An Example



$$D = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix}$$

$$A = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

Vertex order: 1, 2, 3, 4.

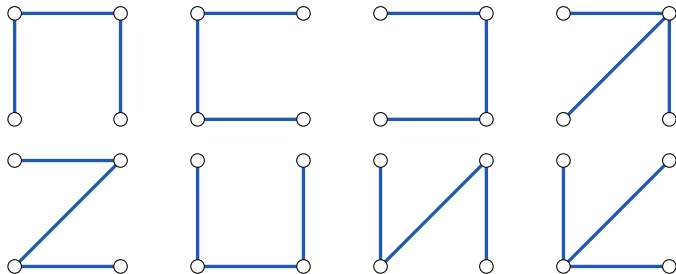
Matrix Tree Theorem: Computing the Answer

$$D - A = \begin{pmatrix} 2 & 0 & -1 & -1 \\ 0 & 2 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{pmatrix}.$$

Delete row 1 and column 1:

$$\#\{\text{spanning trees of } G\} = \det \begin{pmatrix} 2 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{pmatrix} = 16 - 4 - 4 = \boxed{8}.$$

The Eight Spanning Trees



Every drawing keeps the same four vertices.

Summary: Trees

- **Structure of trees:** unique paths, leaves, $n - 1$ edges, and centers.
- **Spanning trees:** every connected graph contains a spanning tree; Kruskal's algorithm finds one of minimum weight.
- **Counting labeled trees:** Prüfer sequences give a bijective proof of Cayley's formula, n^{n-2} .
- **Counting spanning trees:** the Matrix Tree Theorem expresses the answer as any cofactor of $D - A$.

Trails, Circuits, Paths and Cycles